

Instructor Manual & Facilitation Guide

Clinical Data Analysis in R — Phase I: Introduction to R for Clinical Research

Trainers: Vương Mỹ Lượng (Senior Biostatistician, lead trainer) · Bernard Osang'ir (Senior Biostatistician)

Provider: Neudata · #ClearDataClearImpact

Format: 5 evening sessions · every Tuesday, 8:00 PM, 90 minutes (~7.5 contact hours) · 8 September – 6 October 2026 · live-coding workshop

Case study: *Determinants of Hypertension Treatment Uptake among Adults attending Primary Healthcare Facilities* (simulated multicentre cross-sectional study, 1,500 adults, 6 PHC facilities)

How to use this manual

This is a **facilitation guide**, not a textbook. It is written for an instructor (or co-facilitator) teaching the course for the first time. For each day you get: a timing plan, the day's objectives, the teaching through-line, a walkthrough of the live demo (`Scripts/dayN_demo.R`) with expected console output and deliberate "teaching errors," common misconceptions, clinical interpretation notes, anticipated audience questions with model answers, and what success looks like on the exercise (`Practicals/dayN_exercise.R` → `Solutions/dayN_solution.R`).

Suggested reading order before you teach:

1. This manual, front to back.
2. `Data/data_dictionary.md` — the 36 variables and every deliberate data-quality flaw.
3. `References/key_findings.md` — the canonical results you will quote and mark against.
4. The five demo scripts and five solution scripts, run end to end on your own machine the night before.

Convention in this manual: `monospace` = something you type or a file; *italic* = something you say or emphasise verbally; "Surface this" = a moment to slow down and make explicit.

1. Course overview

Aims

Equip clinicians and health researchers with **no prior programming experience** to independently run a complete, reproducible clinical data analysis in R — from importing a messy dataset to reporting adjusted odds ratios in a manuscript-ready Results section. The course is deliberately built around **one realistic study end to end**, so every skill is learned in clinical context, never in the abstract.

Audience

Doctors, nurses, clinical researchers, public-health professionals, residents, master's students, and trial coordinators. **Assume zero coding background.** Many will be anxious about "programming."

Your first job on Day 1 is to lower that anxiety.

The 13 learning objectives

By the end of the course, participants will be able to:

1. Navigate RStudio and run R code from a script and the console.
2. Use objects, vectors, functions, and packages.
3. Import clinical data from CSV and Excel.
4. Inspect a dataset and recognise data-quality problems.
5. Clean categorical text, standardise binary codings, and handle missing-value sentinels.
6. Validate clinical values against plausible physiological ranges.
7. Recode and derive variables (e.g. BMI, BP category) and set factors with correct reference levels.
8. Compute and interpret descriptive statistics (central tendency, spread, frequencies, cross-tabs).
9. Produce a publication-quality "Table 1" and journal-quality figures.
10. Choose and run the correct hypothesis test for a given question.
11. Fit, interpret, and report simple and multivariable logistic regression as odds ratios with 95% CIs.
12. Check model assumptions and discrimination (VIF, linearity, influence, AUC).
13. Work reproducibly (projects, relative paths, saved outputs, `set.seed()`, session info).

Five-session structure (90 minutes each)

Session	Theme (matches the poster & deck)	Recap	Lecture + demo	Exercise	Key deliverable
1	Introduction to R & RStudio	~10 min	~55 min	~20 min	Data imported, first inspection
2	Understanding & Cleaning Clinical Data	~10 min	~55 min	~20 min	<code>analysis_data.rds</code> (the cleaning contract)
3	Descriptive Statistics, Tables & Figures	~10 min	~55 min	~20 min	Table 1 + saved figures
4	Common Medical Statistical Tests	~10 min	~55 min	~20 min	Correct test chosen & interpreted
5	Introduction to Regression & Interpreting Output	~10 min	~55 min	~20 min	Linear + logistic model, forest plot, Results section

- > **Deck-alignment note.** The current slide deck introduces **regression (linear then logistic) in Session 5**²; Session 4 is now tests only. The detailed *Day 4* section further below still contains a logistic-regression primer — when teaching from the current slides, defer that primer to Session 5 (where the deck now places it).

Each session ends with **Q&A** woven through, not bolted on. Days are cumulative: Day 2 produces the clean file that Days 3–5 all read. **If a participant misses Day 2, give them the prepared `analysis_data.rds` so they can keep up.**

2. Pre-course setup checklist

Software (participants — ideally before Day 1)

Direct participants to `References/package_installation_guide.md`. The essentials:

- ☐ Install **R 4.x** (course built with R 4.6.0) from CRAN.
- ☐ Install **RStudio Desktop** from posit.co.
- ☐ Open **RStudio** (not plain R) and confirm the four panes appear.
- ☐ Install all packages in one block (`tidyverse`, `readxl`, `lubridate`, `janitor`, `gtsummary`, `gt`, `broom`, `broom.helpers`, `car`, `pROC`, `scales`).
- ☐ Run the Step 3 verification loop; every line should print `TRUE`.
- ☐ Create an **RStudio Project** rooted at the `Course/` folder (File → New Project → Existing Directory).

> Have a **fallback laptop or two** pre-configured. There will always be someone whose install failed; do not let it derail Day 1.

"Test it works" snippet

Ask everyone to paste this into the Console on Day 1 morning. It confirms R runs, packages load, and the data path resolves:

```
library(tidyverse)
library(readxl)
htn <- read_csv("Data/hypertension_phc_raw.csv")
dim(htn)           # expect 1503 36
cat("Setup OK - R, tidyverse and the data path all work.\n")
```

If `dim(htn)` returns `1503 36`, that participant is ready. (Note: **1503**, not 1500 — the three duplicates are a teaser for Day 2.)

Room / AV requirements

- ☐ Projector or large screen; instructor able to mirror their RStudio.
- ☐ Reliable power (every seat) and Wi-Fi for the first install only.
- ☐ Whiteboard/flip chart for sketching odds ratios, the logit, and folder structure.
- ☐ Ideally a **co-facilitator** to circulate and unblock individuals during exercises.

Distributing the Course folder

Zip and share the entire `Course/` folder (Data, Scripts, Practicals, Solutions, References, Resources, Slides). Participants unzip it and open it **as an RStudio Project**, which makes every relative path (`Data/...`, `Resources/...`) work identically on every machine. **Do not** distribute solutions before each day's exercise — share `Solutions/dayN_solution.R` only after the exercise.

3. General facilitation guidance

Pacing. This is a live-coding course; expect to feel slow. The rate-limiting step is the *slowest typist in the room*, not the fastest learner. Budget time generously and resist the urge to "just paste it." When people type the code themselves they remember it.

The watch-then-do demo pattern. For every concept:

1. *Watch* — you run the demo line, narrate what each part does, read the console output aloud.
2. *Do* — participants type the same line and confirm they get the same output.
3. *Twist* — pose a tiny variation ("now do it for `dbp_mmhg`") so they apply, not copy.

Run demo scripts **line by line** with `Ctrl+Enter` (Windows) / `Cmd+Enter` (Mac). Never run a whole script silently.

Managing a mixed-ability room. Some participants will fly; others will be lost by line three. Tactics: pair fast finishers with strugglers; give fast finishers the "stretch" tasks (each exercise has one); keep a co-facilitator circulating; use a coloured-sticky-note signal ("red = stuck"). Reassure repeatedly that **getting errors is normal and not a sign of failure**.

Encouraging questions. Open every day with: "*There are no silly questions — if you're confused, three other people are too.*" Pause after each demo section and ask a *named, concrete* question ("What does the `$` do here?") rather than the silence-inducing "Any questions?"

Large-font screen sharing. Before Day 1: in RStudio, **Tools** → **Global Options** → **Appearance**, set a large editor font (16–18 pt) and a high-contrast theme. Zoom the console too. People at the back must read the code, not squint.

Handling errors live as teaching moments. You *will* hit errors live — embrace them. When one appears, slow down and say "*Good — let's read the error together.*" Teach the habit of **reading the message, not panicking**. The demos contain deliberate traps (max age 200, `mean()` returning `NA`, complete-case dropping) precisely so the class meets these errors in a controlled way. See the Troubleshooting appendix for the canonical fixes.

4. Daily lesson plans

DAY 1 — Introduction to R & RStudio

Timing (90 min): Recap 10 min · Lecture & live demo 50 min · Guided exercise 25 min · Wrap-up & Q&A 5 min.

Demo script: `Scripts/day1_demo.R` · **Exercise:** `Practicals/day1_exercise.R` · **Solution:** `Solutions/day1_solution.R`

Learning objectives (day): 1, 2, 3, 4 — navigate RStudio; use objects/vectors/functions/packages; import CSV and Excel; take a first critical look at the data.

Through-line. "*R is just a very obedient, very literal calculator that never forgets what you told it.*" Today is about removing fear and getting the clinical data **in**. We do no statistics — we import, look, and discover the data is messy, which sets up Day 2.

Key teaching points.

- The **Script** is what you save and re-run (reproducibility); the **Console** is where results appear. Code lives in the script, not typed once into the console and lost.
- Assignment uses `<-` (read it aloud as *"gets"*): `sbp <- 152` means *sbp gets 152*.
- R is **case-sensitive**: `sbp` and `SBP` are different objects.
- `install.packages()` runs **once** (needs internet); `library()` runs **every session**.
- Relative paths inside an RStudio Project make the analysis portable.

Live-demo walkthrough (day1_demo.R).**1. R as a calculator (lines ~18–21):**

```
2 + 2
mean(c(120, 130, 145, 150)) # mean of four systolic readings
```

Expected: `[1] 4`, then `[1] 136.25`. *Point out the `[1]` prefix — it's just an index, not part of the answer.*

2. Objects and case sensitivity (lines ~31–37):

```
sbp <- 152
sbp + 10
```

Expected `[1] 162`. **Teaching error #1 — surface this:** type `SBP` and let it error `Error: object 'SBP' not found`. *"R is literal: capital S-B-P was never created."*

3. Vectors and logicals (lines ~43–55):

```
sbp_readings <- c(152, 138, 145, 160, 129, 142)
high_bp <- sbp_readings >= 140
sum(high_bp)
```

Expected `sum(high_bp) → [1] 4`. *Explain that `TRUE` counts as 1 — so `sum()` of a logical counts how many are `TRUE`.*

4. Packages (lines ~64–65): `library(tidyverse)` then `library(readxl)`. **Teaching error #2:** if someone typed `library(tidyverse)` before installing, they get there is no package called `'tidyverse'` — the cue to revisit the setup guide.**5. Import (lines ~87–90):**

```
htn <- read_csv("Data/hypertension_phc_raw.csv")
```

Expected console output: a column specification block and `Rows: 1503 Columns: 36`. *Pause here.* The study enrolled 1,500 — *"Why 1503?"* Three duplicate rows. Plant this for Day 2.

6. First look (lines ~99–114):

```
glimpse(htn)
summary(htn$age) # max = 200 -> impossible!
table(htn$sex) # Female, F, female, f ... messy
```

Teaching error #3 / debugging moment: `summary(htn$age)` shows a **Max of 200**, and `table(htn$sex)` shows multiple spellings. Let the room react. *"Before any statistics, R is already telling us the data needs cleaning."*

Common misconceptions & corrections.

- *"I have to memorise all the commands."* No — you look them up; see `References/R_command_reference_sheet.md`. Fluency comes from repetition, not memorisation.
- *"= and <- are the same."* They usually behave the same for assignment, but the course convention is `<-`; `=` is reserved for naming function arguments. Keep it simple and consistent.
- *"The red text means I broke R."* Red is often just a **message** (e.g. the column spec from `read_csv()`), not an error. Teach them to read whether it says `Error`.

Clinical interpretation notes. Even raw inspection is clinically meaningful: a max age of 200 and an sbp of 700 are physiologically impossible and flag data-entry problems. A clinician's domain knowledge is the best data-validation tool.

Suggested audience questions (with answers).

- Q: *Why use R instead of Excel/SPSS?* A: Reproducibility. R records every step as code, so the analysis can be re-run, audited, and shared — essential for clinical research and publication.
- Q: *What's a "tibble"?* A: A modern data frame; it prints neatly (first 10 rows, column types) and behaves predictably.
- Q: *Do I need internet to use R?* A: Only the first time, to install packages. After that it runs offline.

Exercise — what success looks like. Participants independently: load `tidyverse` and `readxl`; import the CSV into `htn` (1503 × 36) and the Excel sheet "data" into `htn_xl` (same dimensions); run `glimpse()` and name 3 numeric and 3 categorical variables; spot **max age = 200** (implausible) and the **multiple spellings of sex**; note stray spaces in `facility`. **Key answer point:** the two data problems they should name are *implausible/impossible values* and *inconsistent category spellings* — exactly the Day 2 agenda.

DAY 2 — Understanding & cleaning clinical data

Timing (90 min): Recap 10 min · Lecture & live demo 50 min · Guided exercise 25 min · Wrap-up & Q&A 5 min.

Demo script: `Scripts/day2_demo.R` · **Exercise:** `Practicals/day2_exercise.R` · **Solution:** `Solutions/day2_solution.R`

Learning objectives (day): 5, 6, 7 — clean text and binaries, handle missing sentinels, validate clinical ranges, recode/derive, set factors with correct reference levels.

Through-line. *"Garbage in, garbage out — 80% of real analysis is cleaning."* Today we turn the messy raw file into one tidy, analysis-ready dataset and **save it**. This script is the **cleaning contract**: Days 3, 4 and 5 all start from `Data/analysis_data.rds`. Emphasise that we **never re-clean by hand later**.

Key teaching points.

- Tell `read_csv` what counts as missing: `na = c("", "NA", "999", "-99")`.
- Remove duplicates with `distinct()`; confirm with `n_distinct(patient_id)`.
- Standardise messy categories with `str_to_lower()` + `case_when()`; write a **reusable helper** (`to_yesno()`) so the same logic applies to every binary.

- Validate against physiological ranges → out-of-range becomes NA.
- **Recompute** BMI from source height/weight rather than trusting the supplied (faulty) column.
- Set **factors with deliberate reference levels** — for binaries, list "No" first so models estimate the odds of the event.

Live-demo walkthrough (day2_demo.R).

1. Import with sentinels (lines ~19–24): `nrow(raw)` → 1503.
2. Duplicates (lines ~30–33):

```
sum(duplicated(raw)) # 3
raw <- distinct(raw)
nrow(raw)           # 1500
```

3. Clean `sex` and binaries (lines ~43–63):

```
table(raw$sex, useNA = "ifany") # now only Female / Male
table(raw$treatment_uptake, useNA = "ifany") # now only Yes / No
```

4. Validation (lines ~70–78): impossible `age`, `sbp_mmhg`, etc. set to NA; `summary()` now shows sane Max values. **Debugging moment:** ask "*Where did age 200 go?*" — it is now counted under NA's.
5. Derive BMI and categories (lines ~84–96). **Teaching error #1:** the supplied `bmi` column is wrong because of the `height_cm = 17` and `weight_kg = 7` errors; recomputing after validation fixes it.
6. Dates (lines ~104–108): `parse_date_time(..., orders = c("ymd", "dmy", "d-b-Y"))`; `sum(is.na(enroll_date))` shows any that failed to parse.
7. Factors (lines ~114–138). **Teaching error #2 — surface this hard:** if you set `treatment_uptake = factor(..., levels = c("Yes", "No"))` (wrong order), every odds ratio on Days 4–5 inverts. The reference level **must** be "No".
8. Save the contract (lines ~153–154):

```
saveRDS(analysis_data, "Data/analysis_data.rds")
write_csv(analysis_data, "Data/analysis_data.csv")
```

Common misconceptions & corrections.

- "*I'll just fix it in Excel.*" That breaks reproducibility and is unauditable. Cleaning belongs in code.
- "*A missing value is zero.*" No — NA means *unknown*, not 0. Recoding 999/-99/blank to NA is the whole point.
- "*Factor order doesn't matter.*" It silently determines the regression reference category — it matters enormously.
- "*|> is different from %>%.*" For this course they behave the same (pipe the left side into the next function); use whichever the script uses.

Clinical interpretation notes. Validation ranges are clinical judgements: age 18–110, SBP 70–260 mmHg, DBP 40–150, height 120–210 cm, weight 30–200 kg. Discuss *why* — these are the bounds of human physiology. The recomputed BMI feeds the standard WHO categories (Underweight/Normal/Overweight/Obese).

Suggested audience questions (with answers).

- Q: *Why save as .rds instead of .csv?* A: .rds preserves R types — crucially the **factor levels and order**. A re-read CSV would lose the reference-level setup.

- **Q: Should I delete rows with any missing data?** A: Not at the cleaning stage. Keep them; the model uses complete cases for its variables only (Day 4). Dropping early throws away usable data.
- **Q: Is `to_yesno()` necessary?** A: It avoids copy-paste errors — one tested function applied to five columns is safer than five hand-edited blocks.

Exercise — what success looks like. A reproducible cleaning pipeline ending in

`analysis_data.rds`: import with `sentinels`; `distinct()` → **1500 rows**; `sex` reduced to Female/Male; `diabetes` and `treatment_uptake` standardised to Yes/No; out-of-range `age/sbp` → NA; BMI recomputed with `bmi_cat`; key variables converted to factors with `treatment_uptake` **reference = "No"**; data saved. **Key answer point:** the count of missing `total_chol_mmol_l` after import (via `sum(is.na(...))`) — confirm they used the `na =` argument so the -99/blank sentinels were caught.

DAY 3 — Descriptive statistics, tables & figures

Timing (90 min): Recap 10 min · Lecture & live demo 50 min · Guided exercise 25 min · Wrap-up & Q&A 5 min.

Demo script: `Scripts/day3_demo.R` · **Exercise:** `Practicals/day3_exercise.R` · **Solution:** `Solutions/day3_solution.R`

Learning objectives (day): 8, 9 — central tendency/spread, frequencies, cross-tabs; publication-quality Table 1 and figures.

Through-line. *"Describe before you test."* Today we summarise the sample numerically and visually and build the **Table 1** every clinical manuscript opens with. Always reload from `analysis_data.rds` — never re-clean.

Key teaching points.

- The **#1 beginner trap**: `mean()/sd()` return NA if *any* value is missing → always `na.rm = TRUE`.
- Mean vs median: when the mean sits above the median, the variable is right-skewed (common for BP, BMI) → report **median (IQR)**.
- `table()` **silently drops** NA → use `useNA = "ifany"` so percentages aren't misleading.
- `prop.table(margin = 1)` = row %, `margin = 2` = column %; **choose the margin that answers your question.**
- `gtsummary::tbl_summary()` builds Table 1; `ggsave(..., dpi = 300)` saves figures for publication.
- Define styling **once** (`course_teal <- "#0D7377"`) and reuse.

Live-demo walkthrough (`day3_demo.R`).

1. The `na.rm` trap (lines ~56–57):

```
mean(analysis_data$sbp_mmhg)           # NA
mean(analysis_data$sbp_mmhg, na.rm = TRUE) # the correct value
```

Debugging moment #1: *"Why did the first one give NA when we have 1500 patients? Because at least one SBP is missing."*

2. Spread and quantiles (lines ~60–66): `median`, `sd`, `IQR`, `quantile()`.

3. Grouped summary by outcome (lines ~101–116): `group_by(treatment_uptake) + across()` — note treated patients tend to be older / higher SBP (early signal).
4. Frequencies (lines ~129–148). **Debugging moment #2:** compare `table(analysis_data$education)` with `table(..., useNA = "ifany")` — the second reveals the blanks.
5. Cross-tabs (lines ~155–163): row vs column percentages; **make the class state the question first** so they pick the right margin.
6. Table 1 (lines ~181–207): `tbl_summary(by = treatment_uptake) |> add_p() |> add_overall() |> bold_labels()`. **Teaching error / fallback:** if `gtsummary` isn't installed, show the base-R `table()/aggregate()` fallback noted in the script.
7. Figures (lines ~229–309): histogram of age, SBP histogram+density, education bar, boxplots by group, SBP-vs-BMI scatter with `geom_smooth(method = "lm")`. Each saved with `ggsave(..., dpi = 300)`.

Common misconceptions & corrections.

- *"A p-value in Table 1 proves causation."* No — Table 1 p-values are descriptive group comparisons, not adjusted effects (that's Days 4–5).
- *"Always report the mean."* Not for skewed data — report median (IQR).
- *"geom_bar() needs me to count first."* No — it counts categories for you; pre-summarising double-counts.
- *"The plot vanished."* `ggsave()` saved the last printed plot; check the `Resources/` folder.

Clinical interpretation notes. The scatter's upward slope (BMI vs SBP) is consistent with obesity as a hypertension risk factor — but **association, not causation**. Skew in SBP/BMI is expected physiologically (a few very high values). Choosing the correct `prop.table` margin is a genuine manuscript pitfall: *"% of diabetics on treatment"* = column %; *"% of treated who are diabetic"* = row %.

Suggested audience questions (with answers).

- *Q: When mean ≠ median, which do I report?* A: For a clearly skewed clinical variable, median (IQR). When they're close, mean (SD) is fine.
- *Q: How do I get Table 1 into Word?* A: `as_flex_table() → flextable::save_as_docx()`, or export HTML/CSV (the script shows both).
- *Q: Why 300 dpi?* A: Journals require it for print-quality figures.

Exercise — what success looks like. Reload the clean data; report mean (SD) age and median (IQR) distance (with `na.rm = TRUE`); frequency tables (n and %) of education and `bp_category`; cross-tab of `treatment_uptake × diabetes` with row %; a **Table 1 stratified by** `treatment_uptake` with `add_p()` covering the listed variables; two saved figures (boxplot of SBP by uptake, education bar) at 300 dpi; Table 1 exported. **Key answer point:** name two characteristics that differ between treated/untreated (e.g. age, insurance, diabetes, education) and judge them clinically plausible.

DAY 4 — Statistical analysis (hypothesis tests + intro to logistic regression)

Timing (90 min): Recap 10 min · Lecture & live demo 50 min · Guided exercise 25 min · Wrap-up & Q&A 5 min.

Demo script: `Scripts/day4_demo.R` · **Exercise:** `Practicals/day4_exercise.R` · **Solution:** `Solutions/day4_solution.R`

Learning objectives (day): 10, 11 (intro) — match question to test; run/interpret t-test, Wilcoxon, ANOVA, chi-square/Fisher, correlation; fit and interpret simple logistic regression as odds ratios.

Through-line. *"Move from describing the data to asking questions of it."* The pivotal idea today is the **analytic population**: treatment uptake only makes sense for people **already diagnosed**, so we filter `htn_diagnosed == "Yes"` (~1,089). *"You cannot take up treatment for a disease you haven't been diagnosed with."*

Key teaching points.

- Define the analytic subset explicitly and once: `dx <- filter(analysis_data, htn_diagnosed == "Yes")`.
- Match the question to the test (the Day 4 recap table):
- two groups, continuous → `t.test()` (or `wilcox.test()` if skewed)
- >2 groups, continuous → `aov()` (or `kruskal.test()`)
- two categoricals → `chisq.test()` (or `fisher.test()` if sparse)
- two continuous → `cor.test()` (Pearson/Spearman)
- binary outcome + drivers → `glm(..., family = binomial)` → odds ratios
- **Plot before you test.** Histogram + Q-Q first; Shapiro-Wilk is over-powered at $n \approx 1,089$.
- Odds ratios = `exp(coef())`; CIs = `exp(confint())`; rescale continuous predictors to meaningful units (e.g. per 10 years).

Live-demo walkthrough (day4_demo.R).

1. Build the subset (lines ~41–49): `nrow(dx) ≈ 1089`; `table(dx$treatment_uptake)`. **Debugging moment #1 — surface this:** running on all 1500 mixes in undiagnosed people for whom the outcome is undefined, biasing everything.
2. Normality (lines ~63–83): `hist`, `qqnorm/qqline`, then `shapiro.test()`. *"With ~1089 rows Shapiro flags trivial departures — trust the Q-Q plot."*
3. t-test / Wilcoxon (lines ~104–114): `t.test(age ~ treatment_uptake, data = dx)` — read the two means, the CI for the difference, the p-value. Expect older age among treated.
4. ANOVA + Tukey (lines ~129–139): `aov(age ~ education)` then `TukeyHSD()`. *"ANOVA says some groups differ; Tukey says which."*
5. Chi-square / Fisher (lines ~151–165): build `table(dx$treatment_uptake, dx$diabetes)`, run `chisq.test()`, **check** `$expected`, fall back to `fisher.test()` if any expected < 5.
6. Correlation (lines ~178–181): Pearson vs Spearman for SBP vs BMI. *"In big samples a tiny r can be 'significant' yet clinically trivial — report r, not just p."*
7. Simple logistic regression (lines ~202–229):

```
m_diab <- glm(treatment_uptake ~ diabetes, data = dx, family = binomial)
exp(coef(m_diab)); exp(confint(m_diab))
m_age <- glm(treatment_uptake ~ age, data = dx, family = binomial)
exp(coef(m_age)["age"] * 10) # OR per decade
```

"OR > 1 = higher odds of uptake; a CI excluding 1 = significant."

8. Brief multivariable + `broom` (lines ~241–264). **Debugging moment #2:** `nobs(m_multi) < 1089` because `glm()` uses **complete cases** — any missing value in any model variable drops that row silently. `tidy(m_multi, exponentiate = TRUE, conf.int = TRUE)` gives the report-ready table.

Common misconceptions & corrections.

- *"Significant = important."* No — significance is about evidence against the null; clinical importance is the effect size (OR, mean difference, r).
- *"A small Shapiro p means I can't use a t -test."* Not at this sample size; judge the Q-Q plot.
- *"The OR per year of age is tiny so age doesn't matter."* It's per **one year**; rescale to a decade to see the real effect.
- *"glm used all my patients."* Check `nobs()` — complete-case analysis quietly reduces n .

Clinical interpretation notes. Diabetics attending PHC are sicker and more engaged with care, so a higher uptake ($OR > 1$) is plausible. Phrase ORs clinically: *"Patients with diabetes had about X times the odds of treatment uptake versus those without ($OR\ X.X$, 95% CI $a-b$)."* Remind them: chi-square gives a p -value, not an effect size — the OR does.

Suggested audience questions (with answers).

- Q: *t-test or Wilcoxon?* A: If the variable is roughly symmetric in a large sample, t -test; if clearly skewed or small n , Wilcoxon. When they agree, report the t -test.
- Q: *Why logistic and not linear regression?* A: The outcome is binary (Yes/No); logistic models the log-odds, giving interpretable odds ratios.
- Q: *Why is the reference "No"?* A: We set it on Day 2 so the model estimates the odds of **uptake** (the event of interest).

Exercise — what success looks like. Restrict to `dx` (≈ 1089); decide t -test vs Wilcoxon for age by uptake and interpret; chi-square of uptake $\times$ diabetes; simple logistic OR for diabetes with CI and a one-sentence clinical reading; OR for age per year and per decade (OR^{10}); a multivariable model (`age + sex + diabetes + residence + health_insurance`) tidied with `broom`. **Key answer point:** they should list the significant determinants with directions, broadly matching `key_findings.md` (diabetes $\uparrow$, insurance $\uparrow$, urban $\uparrow$, older age $\uparrow$). Mark on **correct method and interpretation**, not second-decimal matches.

DAY 5 — Regression modelling & reproducibility (capstone)

Timing (90 min): Recap 10 min · Lecture & live demo 50 min · Guided exercise 25 min · Wrap-up & Q&A 5 min.

Demo script: `Scripts/day5_demo.R` · **Exercise:** `Practicals/day5_exercise.R` · **Solution:** `Solutions/day5_solution.R`

Learning objectives (day): 11 (full), 12, 13 — fit/report a multivariable model; confounding and interaction; diagnostics (VIF, linearity, influence, AUC); reproducibility.

Through-line. *"Build the model from clinical knowledge, not from p -values; then report it reproducibly."* This is the capstone: one pre-specified multivariable model, properly diagnosed, presented as adjusted ORs with a forest plot, all reproducible.

Key teaching points.

- **Pre-specify** predictors from clinical knowledge and the literature — avoid data dredging and uncritical stepwise selection.
- **Confounding:** compare crude vs adjusted OR for residence; a $>10\%$ shift signals confounding.

- **Interaction (effect modification):** add `diabetes:age`, compare nested models with an LRT (`anova(..., test = "LRT")`); expect non-significant → keep the simpler model.
- **Diagnostics:** `car::vif()` (>5 worth a look, >10 serious); linearity on the **logit** (loess check); Cook's distance via `broom::augment()`; **AUC** via `pROC` (0.7–0.8 acceptable).
- **Reproducibility:** `set.seed()`, relative paths, saved outputs, `sessionInfo()`, and the path to R Markdown/Quarto one-click reports.

Live-demo walkthrough (day5_demo.R).

1. Load + subset (lines ~36–49): `dx`, confirm levels(`dx$treatment_uptake`) is `c("No", "Yes")` (reference = No).
2. Full pre-specified model (lines ~65–77): `treatment_uptake ~ age + sex + education + residence + diabetes + family_history_htn + health_insurance + knowledge_score + distance_to_facility_km`.
3. Confounding (lines ~86–90): crude vs adjusted OR for `residenceUrban` — note the shift.
4. Interaction (lines ~105–114): `anova(model_full, model_interax, test = "LRT")` — **expect $p > 0.05$** , keep `model_full`. **Teaching point:** with the interaction in, the `diabetes` main effect is the effect *at age 0* — meaningless; don't interpret main effects under a retained interaction.
5. Stepwise caution (lines ~130–144): `step()` shown then **rejected** in favour of the clinical model. Say the cautions out loud.
6. Diagnostics (lines ~153–200): `vif()`; loess linearity check for `knowledge_score`; Cook's distance with cutoff $4/n$; **AUC ≈ 0.71** .
7. Report (lines ~207–259): `tidy(model_final, exponentiate = TRUE, conf.int = TRUE)`; `tbl_regression()`; forest plot on a **log scale** with reference line at OR = 1, saved to `Resources/`.
8. Reproducibility (lines ~271): `sessionInfo()`; mention writing it to `References/session_info.txt`.

Expected canonical results (from `key_findings.md` — for marking). Analytic sample 1,089 diagnosed; **992 complete cases**; uptake $\approx 47\%$. Adjusted ORs: diabetes **3.56**, insurance **2.05**, family history **1.91**, urban **1.87**, education trend **1.96**, age **1.03/yr**, knowledge **1.10/pt**, male sex **0.74**, distance **0.98 (NS)**; **AUC ≈ 0.71** . Decimals shift trivially across versions — mark on method and interpretation.

Common misconceptions & corrections.

- *"Drop every non-significant variable."* No — for an **explanatory** study, keep clinically chosen confounders even if $p > 0.05$; dropping them reintroduces bias.
- *"Stepwise/AIC finds the 'true' model."* It optimises fit-vs-complexity, produces optimistic CIs, and isn't reproducible across datasets. Reserve it mostly for prediction.
- *"A flagged influential point should be deleted."* Inspect it first; report a sensitivity analysis if results change.
- *"Higher AUC is always the goal."* For an explanatory determinants study, AUC summarises discrimination, not the validity of the ORs; 0.71 is acceptable here.

Clinical interpretation notes. Diabetes shows the **largest effect** (OR ~ 3.56) but the **widest CI** — a smaller subgroup means lower precision; a perfect teaching point on effect size vs precision. The residence crude-vs-adjusted comparison demonstrates mild confounding. Distance trends protective

but is **non-significant** after adjustment — a reminder that an expected direction is not the same as statistical significance.

Suggested audience questions (with answers).

- *Q: Why keep distance if it's not significant?* A: It's a pre-specified access variable; reporting its (non-significant) adjusted OR is honest and informative.
- *Q: What does AUC = 0.71 mean?* A: Given a random treated and untreated patient, the model ranks the treated one higher 71% of the time — acceptable discrimination.
- *Q: How do I make this fully reproducible?* A: RStudio Project + relative paths + `set.seed()` + saved outputs + `sessionInfo()`, ideally knitted from an R Markdown/Quarto document.

Exercise — what success looks like. Fit the full pre-specified model on `dx`; `vif()` (no value > 5); a publication-ready OR table (`tbl_regression` or `broom::tidy`) saved to `Resources/`; a saved forest plot; AUC via `pROC` (≈ 0.71); a 150–250-word plain-English Results section naming the significant determinants with adjusted ORs and 95% CIs plus one sentence on discrimination. **Key answer point:** largest effect = diabetes; widest CI = diabetes — width reflects lower precision from the smaller subgroup. Compare against `References/results_section_draft.txt`.

5. Troubleshooting appendix — common R errors clinicians hit

Symptom / message	Likely cause	Fix
could not find function "glimpse" / "tbl_summary"	Package not loaded this session	<code>library(tidyverse) / library(gtsummary)</code> before use. <code>install.packages()</code> once; <code>library()</code> every session.
there is no package called 'X'	Never installed, or name typo	<code>install.packages("X");</code> check spelling/case matches <code>library(X)</code> .
cannot open file 'Data/...': No such file or directory	Wrong working directory / not in the Project	Open the Course/ RStudio Project ; check <code>getwd()</code> ; use relative paths, never <code>setwd("C:/Users/...")</code> .
Error: object 'sbp' not found	Object never created, or case mismatch (SBP vs sbp)	Re-run the line that creates it; match capitalisation exactly.
A summary returns NA for a numeric variable	A missing value propagates through mean/sd	Add <code>na.rm = TRUE</code> .
A category is missing from a frequency table	<code>table()</code> drops NA silently	<code>table(x, useNA = "ifany")</code> .
Odds ratios are inverted / "protective when expected harmful"	Factor reference level wrong order	Set levels with "No"/"Rural" first: <code>factor(x, levels = c("No", "Yes"))</code> .
Error: unexpected '=' or argument-vs-assignment confusion	Used = where <- was meant (or vice-versa)	Use <- to create objects; = only for function arguments.
Model <code>nobs()</code> lower than expected	<code>glm()</code> uses complete cases; missing values drop rows	Check missingness; decide on imputation or report complete-case n.
<code>chisq.test</code> warning "approximation may be incorrect"	Expected cell counts < 5	Use <code>fisher.test()</code> .
Table export to PNG fails / "Chrome not found"	<code>gt</code> PNG export needs Chrome	Export <code>.html/.csv</code> instead (scripts fall back automatically).

Symptom / message	Likely cause	Fix
<code>read_csv</code> shows 1503 rows	Duplicates present (by design)	Day 2: <code>distinct()</code> → 1500.

> **The meta-skill:** teach participants to **read the error message aloud** and locate the offending object/file/function before changing anything. Most errors here are one of: package not loaded, wrong path, wrong case, or a missing-value/factor-level issue.

6. Assessment overview

Final take-home assignment. Participants independently reproduce the full determinants analysis on `analysis_data.rds`, restricted to diagnosed hypertensives: fit the pre-specified multivariable logistic model, check multicollinearity (VIF), present adjusted ORs with 95% CIs (publication-ready table), produce a forest plot, report model discrimination (AUC), and write a 150–250-word Results section in plain clinical English. This mirrors the **Day 5 in-class exercise** (`Practicals/day5_exercise.R`), which is explicitly the practice run for the assignment; canonical answers are in `Solutions/day5_solution.R` and `References/key_findings.md`.

Marking. A detailed marking guide is provided **separately** to instructors. Mark on **correct method and sound interpretation**, not exact second-decimal matches — CIs and directions are stable across R/package versions; precise decimals are not. Reward: correct analytic population (diagnosed only), correct reference levels, adjusted (not crude) ORs, honest reporting of non-significant pre-specified variables, and a clinically literate Results paragraph.

7. Adapting the course for shorter / longer formats

Shorter (½-day or 1-day taster). Compress to Days 1–3: get data in, clean it, produce Table 1 and figures. Supply the prepared `analysis_data.rds` so cleaning can be demonstrated rather than fully performed. Drop hypothesis tests and regression, or give only a 20-minute "here's where this leads" preview of odds ratios.

Standard (this 5-day course). As written — one concept per day, cumulative, ~2 hours/day.

Longer (7–10 days or a semester module). Expand with: a dedicated data-cleaning lab on a second messy dataset; tidyverse `dplyr` verbs in depth; data visualisation as its own day; missing-data handling and multiple imputation; survival analysis or mixed/multilevel models for the multicentre clustering; and a full R Markdown/Quarto reporting day culminating in a knitted manuscript. Add formative quizzes between days and a peer code-review session before the final assignment.

Prepared for instructors of Clinical Data Analysis in R — Phase I · Trainers: Vương Mỹ Lượng (Senior Biostatistician, lead trainer) & Bernard Osang'ir (Senior Biostatistician) · Neudata · #ClearDataClearImpact